

<code>void pop_front()</code>	<code>void splice(iterator pos, list& L, iterator i)</code>
<code>void pop_back()</code>	<code>void splice(iterator pos, list& L, iterator f, iterator l)</code>
<code>void swap(list&)</code>	<code>size_type size() const</code>
<code>iterator insert(iterator pos, const T& x)</code>	<code>size_type max_size() const</code>
<code>template <class InputIterator> void insert(iterator pos, InputIterator f, InputIterator l)</code>	<code>bool empty() const</code>

Among these, operations such as 'size', 'empty', 'max_size', 'front', 'back', etc, are query operations, while 'insert', 'splice', 'clear', 'merge', etc, are update operations. Consider the 'size' operation. If it was implemented by traversing the list and counting the number of elements present in the list when the operation was invoked, then 'size' has complexity that's linear to the size of 'list', i.e., if the list has n elements in it, then 'size' will take $O(n)$. On the other hand, if the number of objects is maintained internally by the list ADT, and various update operations that change the number of objects update this count, then the 'size' operation will take $O(1)$. So, in order for programmers to know whether 'size' is a costly operation or not, the programmers need to know the internal implementation details of the 'size' operation. If they do not know this, they can end up using 'size' in a hot loop, where they will incur the penalty of $O(n)$ for each loop iteration. In this case, there is a cost trade-off between abstraction and knowing the internal implementation of the ADT. This is known as the 'abstraction penalty'.

The abstraction penalty creeps in without the programmer's knowledge. For instance, programmers decide to re-factor their code so that they can move a piece of functionality into a separate function, which is called from multiple places, instead of having the code duplicated in many places. However, they may find that the compiler does not end up in-lining the code, but ends up having an adverse performance impact.

Grouping variables into structures or objects is another common cause of abstraction penalty. For example, consider that you have two local variables `int x, int y`, which are supposed to represent the x and y co-ordinates of a point. Now you have decided to create a new struct 'Point', which contains 'x' and 'y' as members. It is possible that in certain compilers, and at certain optimisation levels, the members 'x' and 'y' will not be placed in registers. This can result in an adverse performance impact. Similarly, placing 'x' and 'y' in a struct may prevent them from being passed through registers when a function call is made, again resulting in an abstraction penalty. While, in theory, compilers and other tools are supposed to eliminate the

abstraction penalty so that programmers can write abstract code without any performance penalties, in reality this is not true. Programmers end up paying the penalty due to software abstraction. If you are interested in knowing how much abstraction penalty is associated with your standard STL implementation and compiler, you can run the abstraction penalty benchmark written by Alexander Stepanov (of C++ STL fame), which is available at: <http://www.stepanovpapers.com/AbstractionPenaltyBenchmark.cpp>. It is a very simple benchmark, which adds 2000 doubles in an array 25,000 times, but performs it in 11 different ways.

While the abstraction penalty differs for different STL implementations and compilers, the point to note is that there is a realistic penalty associated with abstraction, and programmers need to be aware of it. By now, you may be wondering why are we spending time on the abstraction penalty when the discussion ought to be about software bloat? Well, bloat and the abstraction penalty arise from the same root cause of making software abstract. Abstraction is useful for improving programmer productivity, but can cause bloat and container inefficiencies, which can result in poor performance. In the next column, we will discuss what you as a programmer can do, to reduce the abstraction penalty your software may incur.

My 'must-read book' for this month

This month's must-read book suggestion comes from our reader Kavya. She suggests the book 'Web 2.0 Architectures: What Entrepreneurs and Information Architects Need to Know' by James Governor, Dion Hinchcliffe, Duane Nickull. According to Kavya, "This is a great book if you want to understand how Web 2.0 evolved, what the various architectural patterns associated with different Web 2.0 solutions are, and how you can apply these patterns to the further evolution of the Web." Thank you, Kavya, for the recommendation.

If you have a favourite programming book or article that you think is a must-read for every programmer, please do send me a note with the book's name, and a short write-up on why you think it is useful, so that I can mention it in the column. This would help many readers who want to improve their coding skills.

If you have any favourite programming puzzles that you would like to discuss on this forum, please send them to me, along with your solutions and feedback, at sandyasm_AT_yahoo_DOT_com. Till we meet again next month, happy programming, and here's wishing you the very best! 

By: Sandya Mannarswamy

The author is an expert in systems software and is currently working as a researcher in IBM India Research Labs. Her interests include compilers, multi-core technologies and software development tools. If you are preparing for systems software interviews, you may find it useful to visit Sandya's LinkedIn group 'Computer Science Interview Training India' at <http://www.linkedin.com/groups?home=&gid=2339182>.